

Cahier de charges

Cahier des charges — Projets de groupe : "Architectures de réseaux de neurones"

Objectif global : répartir les 10 sujets entre 10 groupes (1 sujet par groupe) pour produire un exposé scientifique + livrables pratiques (rapport, diaporama, code reproductible et démonstration, vidéo explicative) qui montrent compréhension théorique, implémentation et évaluation expérimentale.

Sujet A : Architectures Transformer : mécanisme d'attention et applications

- ✓ **Objectifs :** expliquer l'architecture Transformer, le mécanisme d'auto-attention (self-attention), l'attention multi-tête, les positional encodings et les blocs encoder/decoder ; présenter les techniques d'optimisation (AdamW, learning rate scheduling — warmup & cosine/linear decay) et les méthodes de régularisation pertinentes (dropout, weight decay, label smoothing, layer norm), puis comparer stratégies et variants sur tâches sequence-to-sequence et classification textuelle.

2. Sujet B : Réseaux convolutifs (CNN) et vision : architectures modernes

- **Objectifs :** présenter CNN, couches conv, pooling, architectures (AlexNet, VGG, ResNet, EfficientNet), et implémenter un pipeline de classification d'images.
- **Livrables :** rapport, diapo, notebook avec entraînement sur CIFAR-10 ou subset de ImageNet, visualisation de filtres/feature maps.
- **Contenu :** théorie conv; architectures historiques; transfert learning; visualisations; évaluation et comparaisons.
- **Spécificités techniques :** utiliser transfer learning, pretrained models, augmentation d'images, GPU recommandé.
- Planning, ressources (papiers, tutoriels), critères d'évaluation et risques (compute) similaires au précédent.

3. Sujet C : Réseaux récurrents (RNN) et LSTM/GRU pour séquences

- **Objectifs :** expliquer RNN, problèmes de gradient, LSTM/GRU, applications (texte, séries temporelles), et implémenter une tâche de prédiction de séquences.
- Contenu : RNN vanilla, LSTM gates, GRU, cas d'usage, techniques d'entraînement (teacher forcing, gradient clipping).
- Spécifs : datasets (IMDB, PTB, série temporelle), PyTorch/TensorFlow.
- Risques : training instable, long.

4. Sujet D — Architectures Transformer et attention

- Objectifs : présenter le Transformer (self-attention, multi-head, position encoding), fine-tuning pour NLP, et implémenter un petit modèle Transformer sur classification textuelle.
- Contenu : attention mécanisme, encoder/decoder, applications (BERT, GPT), optimisation et tokenisation.
- Spécifs : Hugging Face Transformers, datasets (SST-2, AG News), GPU fortement recommandé.
- Risques : modèles lourds — proposer fine-tuning de modèles pré-entraînés plutôt que entraînement from scratch.

5. Sujet E : Réseaux résiduels (ResNet) et entraînement de réseaux profonds

- 🚩 Objectifs : expliquer le problème de dégradation avec profondeur, skip connections, architecture ResNet, variantes et justification mathématique, implémenter et comparer profondeurs différentes.
- 🚩 Contenu : bloc résiduel, batch norm, initialisation, optimizers, étude d'ablation.
- 🚩 Spécifs : PyTorch pretrained models, techniques d'accélération.

Fiche de projets : Deep Learning

⚠ Risques : tuning de profondeur.

6. Sujet F : Réseaux génératifs : Autoencodeurs, VAE et GAN

- Objectifs : présenter autoencodeurs, VAE (théorie ELBO), GAN (generator/discriminator, loss instable), et implémenter un VAE et un GAN pour génération d'images simples.
- Contenu : architectures, pertes, stabilité d'entraînement, métriques (FID, IS), améliorations (WGAN, gradient penalty).
- Spécifs : expérimentation visuelle, gestion instabilité.
- Risques : GANs difficiles à former — proposer variantes stables.

7. Sujet G : Réseaux de graphes (GNN) : théorie et applications

- ❖ Objectifs : introduction aux GNN (GCN, GraphSAGE, GAT), propagation de messages, applications (classification de nœuds, prédiction de liens, chimie), implémentation sur dataset simple (Cora, PubMed, MoleculeNet).
- ❖ Contenu : message passing, pooling graph, embeddings, cas d'usage.
- ❖ Spécifs : bibliothèques spécialisées, structures de données graph.
- ❖ Risques : comprendre structure des graph.

8. Sujet H : Deep Reinforcement Learning (Deep RL) et intégration réseaux

- Objectifs : exposer DQN, Policy Gradient, Actor-Critic, intégrer réseaux profonds pour approx fonctions de valeur/politiques, implémenter DQN sur un environnement OpenAI Gym simple.
- Contenu : MDP, exploration-exploitation, replay buffer, target network, stabilité et hyperparams.
- Spécifs : gym, stable-baselines3 optionnel, GPU/CPU selon l'environnement.
- Risques : durée d'entraînement, variance des résultats.

9. Sujet I : Réseaux modulaires et réseaux à capsules (CapsNet)

- **Objectifs** : expliquer architecture CapsNet (capsules, routing), comparer avec CNN, implémenter une version simple pour MNIST/affNIST et visualiser reconstructions.
- Contenu : définition capsules, routing algos (dynamic/EM), loss (margin + reconstruction), variantes récentes.
- Spécifs : computation cost, implémentation recommandée en PyTorch.
- Risques : routing lent — proposer paramétrage réduit.

10. Sujet J : Architectures pour edge/embedded AI : modèles légers et optimisation

- ✓ Objectifs : présenter modèles compacts (MobileNet, EfficientNet, SqueezeNet), techniques d'optimisation (pruning, quantization, distillation), et déployer un modèle optimisé sur CPU/Edge (simulateur).
- ✓ Contenu : architecture mobile-friendly, trade-offs accuracy/latency, pipeline de post-training quantization, exemples pratiques.
- ✓ Spécifs : outils (TensorFlow Lite, ONNX, PyTorch quantization), mesurer latence sur CPU.
- ✓ Risques : contraintes hardware, besoins d'outilchain.

Consignes générales communes à tous les groupes

- Taille des groupes : 2 étudiants.
- Langue : français (rapport + Vidéo + présentation).
- Formats : rapport PDF (8–12 pages selon sujet), diaporama (PDF/PowerPoint), code (notebook + scripts), requirements.txt et README.
- Réutilisabilité : code bien commenté et licences claires (MIT ou équivalent).
- Reproductibilité : fixer seeds, versions de libs, expliquer exécution pas-à-pas.
- Présentation orale : 10–15 minutes + 5 minutes Q&A.
- Délais : remise des livrables et présentation durant la semaine finale du module.

Fiche de projets : Deep Learning

Barème d'évaluation (identique pour tous)

- Contenu théorique et rigueur (30%)
- Implémentation et code reproductible (25%)
- Qualité des expériences et interprétation (25%)
- Qualité des supports et présentation orale (10%)
- Respect du cahier des charges et livraison (10%)

Modèle de planning détaillé (exemple 4 semaines)

- Etape 0 : formation des groupes, distribution des sujets, briefing.
- Etape 1 : recherche bibliographique, plan détaillé, dépôt d'un sommaire et bibliographie.
- Etape 2 : première implémentation, premières expériences, checkpoint intermédiaire (livrable partiel).
- Etape 3 : expériences complètes, visualisations, itérations, préparation diapo.
- Etape 4 : rédaction finale, répétition, dépôt des livrables, présentations.

Ressources communes recommandées

- **Frameworks** : PyTorch, TensorFlow, scikit-learn, PyTorch Geometric / DGL pour GNN, Hugging Face Transformers.
- **Datasets** : MNIST, CIFAR-10, UCI, Cora/PubMed, OpenAI Gym environnements, MoleculeNet, SST/AG News.
- **Outils** : Git/GitHub pour versioning, Google Colab ou GPU university cluster, TensorBoard, matplotlib/seaborn.
- **Lectures** : articles fondateurs (AlexNet, ResNet, Transformer, Sabour CapsNet, Goodfellow GAN, Kipf GCN), tutoriels officiels.